



Guide de Configuration iOS

Application SIGN — Préparation Production

BLK-05	DEVELOPMENT_TEAM — Signing Xcode
AS-03	GoogleService-Info.plist — Firebase iOS

Application	SIGN — Gestion & Signature Électronique
Bundle ID	com.signapp.sign
Plateforme	iOS 13.0+
Flutter	3.35.6 stable
Date	07 June 2026
Confidentiel	Usage interne uniquement

Table des matières

01	Prérequis obligatoires	3
02	BLK-05 — Configurer le Signing Xcode	4
2. 1	Créer un compte Apple Developer	4
2. 2	Ouvrir le projet dans Xcode	4
2. 3	Configurer Signing & Capabilities	5
2. 4	Vérifier le certificat	6
2. 5	Résolution des erreurs courantes	7
03	AS-03 — Configurer Firebase iOS	8
3. 1	Créer l'app iOS dans Firebase	8
3. 2	Télécharger GoogleService-Info.plist	9
3. 3	Ajouter le fichier dans Xcode	10
3. 4	Vérifier l'intégration Firebase	11
04	Build Release iOS	12
05	Checklist finale avant App Store	13

01 — Prérequis obligatoires

Avant de commencer la configuration iOS, assurez-vous d'avoir tous les éléments suivants. L'absence de l'un d'eux bloquera le déploiement.

Matériel

- Un Mac (macOS 13 Ventura minimum recommandé)
- iPhone physique pour les tests (recommandé)

Logiciels

- Xcode 15+ installé depuis le Mac App Store
- Flutter SDK 3.35.6 configuré
- Compte Apple Developer actif (99 \$/an)
- Compte Firebase avec projet existant

Informations

- Bundle ID : **com.signapp.sign**
- Nom de l'app : **SIGN**
- Dépôt git cloné sur le Mac

■■ Important — Système requis

La configuration Xcode et la soumission App Store nécessitent **impérativement un Mac**. Ces étapes ne peuvent pas être effectuées sous Windows.

Si vous développez sous Windows, transférez le dossier du projet sur un Mac ou utilisez un service de build cloud (Codemagic, Bitrise).

02 — BLK-05 : Configurer le Signing Xcode (DEVELOPMENT_TEAM)

Sans **DEVELOPMENT_TEAM** configuré, Xcode ne peut pas signer l'IPA. Il est impossible de faire un build release ou de soumettre à l'App Store. C'est la correction la plus critique pour le déploiement iOS.

2.1 — Créer / Vérifier le compte Apple Developer

Un abonnement Apple Developer Program est requis pour signer et distribuer l'app.

1

■ Aller sur developer.apple.com

Ouvrir <https://developer.apple.com/programs/enroll/>

Se connecter avec votre Apple ID existant.

2

■ S'inscrire comme individuel ou organisation

Individual (vous seul) : 99 USD/an — approuvé immédiatement.

Organization : nécessite un numéro DUNS — délai 3-5 jours ouvrés.

3

■ Confirmer le paiement

Le compte est actif immédiatement après paiement (individuel).

Vous recevez un email de confirmation avec votre Team ID (10 caractères alphanumériques).

URL	https://developer.apple.com
Prix	99 USD/an (renouvellement annuel)
Team ID	Format : ABC1234567 (10 caractères — visible dans Membership)
Délai	Individuel : immédiat Organisation : 3-5 jours ouvrés

2.2 — Ouvrir le projet dans Xcode

■ Toujours ouvrir .xcworkspace, jamais .xcodeproj

Utilisez **ios/Runner.xcworkspace** (et non **Runner.xcodeproj**). Le fichier .xcworkspace intègre CocoaPods (dépendances Firebase, etc.). Ouvrir .xcodeproj seul provoque des erreurs de compilation.

1

■ Ouvrir le terminal sur Mac

Naviguer vers le répertoire du projet :

```
cd ~/path/to/sign_application
```

Terminal Mac

```
# Ouvrir directement depuis le terminal :  
open ios/Runner.xcworkspace  
  
# Ou double-cliquer depuis le Finder sur :  
ios/Runner.xcworkspace
```

2**■ Vérifier que le projet se charge correctement**

Dans le panneau gauche de Xcode, vérifier la structure :

Runner (projet principal)

Pods (dépendances CocoaPods)

RunnerTests (tests)

2.3 — Configurer Signing & Capabilities

C'est l'étape centrale. Xcode va automatiquement créer les certificats et profils de provisioning nécessaires.

1

■ Sélectionner la cible Runner

Dans Xcode, cliquer sur "Runner" dans le panneau gauche (arbre de projet).
Sélectionner la cible "Runner" (icône bleue) dans la liste centrale.

2

■ Aller dans Signing & Capabilities

Cliquer sur l'onglet "Signing & Capabilities" dans la barre principale.

3

■ Activer Automatically manage signing

Cocher la case "Automatically manage signing".
Xcode s'occupera des certificats et profils automatiquement.

4

■ Sélectionner votre Team

Cliquer sur le menu déroulant "Team".
Sélectionner votre compte Apple Developer (format: Prénom Nom ou Organisation).
Si votre compte n'apparaît pas : Xcode → Preferences → Accounts → + → Apple ID.

5

■ Vérifier le Bundle Identifier

S'assurer que "Bundle Identifier" affiche : com.signapp.sign
(Déjà configuré dans project.pbxproj par le code de l'audit)

■ Configuration automatique

Après avoir sélectionné votre Team et activé "Automatically manage signing", Xcode crée automatiquement :

- Un certificat de développement et de distribution (dans votre trousseau)
- Un profil de provisioning (lié au Bundle ID)
- Enregistre le device connecté (pour tests)

2.4 — Vérifier et gérer les certificats

1

■ Ouvrir le Keychain Access (Trousseau d'accès)

Sur Mac : Spotlight → "Trousseau d'accès" (Keychain Access)

Aller dans : Mes certificats

Vérifier la présence de "Apple Distribution: [votre nom]"

2

■ Dans Xcode — gérer les certificats

Xcode → menu Xcode → Settings → Accounts

Sélectionner votre Apple ID

Cliquer "Manage Certificates..."

Cliquer "+" → "Apple Distribution" si absent

3

■ Vérifier dans le portail Apple Developer

Aller sur <https://developer.apple.com/account>

Certificates, IDs & Profiles → Certificates

Vérifier qu'un certificat "Apple Distribution" est actif (vert)

Terminal Mac — Vérification certificat

Vérifier les certificats disponibles depuis le terminal :

```
security find-identity -v -p codesigning
```

Résultat attendu (exemple) :

```
1) ABCD1234... "Apple Distribution: Votre Nom (TEAM1234)"
```

```
1 valid identities found
```

2.5 — Résolution des erreurs courantes

Erreur	Cause	Solution
"No accounts with iTunes Connect access"	Votre Apple ID n'est pas inscrit au Developer Program.	S'inscrire sur developer.apple.com/programs
"Failed to create provisioning profile"	Le Bundle ID com.signapp.sign n'est pas encore enregistré.	Apple Developer → Identifiers → "+" → App ID → com.signapp.sign
"Certificate has been revoked"	Le certificat existant a été révoqué.	Xcode → Settings → Accounts → Manage Certificates → créer nouveau
"Xcode couldn't find a profile matching"	Le profil de provisioning est manquant ou périmé.	Xcode → Product → Clean Build Folder, puis re-build

03 — AS-03 : Configurer Firebase iOS (GoogleService-Info.plist)

Sans **GoogleService-Info.plist**, l'appel `Firebase.initializeApp()` provoque un crash fatal au démarrage de l'app iOS. Ce fichier contient la configuration unique de votre projet Firebase pour la plateforme iOS.

■ Crash garanti sans ce fichier

L'app iOS crashe immédiatement au lancement (avant même d'afficher le splash screen) si `GoogleService-Info.plist` est absent de `ios/Runner/`.

Le code a été protégé par un `try/catch` dans `main.dart` pour éviter le crash complet, mais Firebase et Crashlytics ne seront PAS actifs sans ce fichier.

3.1 — Créer l'app iOS dans le projet Firebase

Si votre projet Firebase existe déjà (pour Android), vous devez y ajouter une application iOS avec le bon Bundle ID.

1

■ Ouvrir la Firebase Console

Aller sur <https://console.firebase.google.com>

Cliquer sur votre projet existant "sign-app" (ou similaire).

2

■ Ajouter une application iOS

Dans la vue d'ensemble du projet, cliquer "+ Add app".

Sélectionner l'icône iOS (pomme).

3

■ Renseigner le Bundle ID iOS

iOS bundle ID : `com.signapp.sign` (EXACTEMENT — respecter la casse)

App nickname : SIGN iOS (facultatif, pour vous repérer)

App Store ID : laisser vide pour l'instant

Cliquer "Register app".

Bundle ID iOS	<code>com.signapp.sign</code> ← OBLIGATOIRE, exact
Bundle ID Android	<code>com.signapp.sign_application</code> ← différent
Nickname	SIGN iOS (optionnel)
App Store ID	Laisser vide (remplir après publication)

3.2 — Télécharger GoogleService-Info.plist

1

■ Télécharger le fichier de configuration

Après "Register app", Firebase affiche automatiquement le bouton de téléchargement.

Cliquer "Download GoogleService-Info.plist".

Le fichier se télécharge dans votre dossier Téléchargements.

2

■ Vérifier le contenu du fichier

Ouvrir le fichier téléchargé (double-clic → TextEdit ou éditeur).

Vérifier que BUNDLE_ID = com.signapp.sign

Contenu attendu de GoogleService-Info.plist

```
CLIENT_ID
12345-abcdef.apps.googleusercontent.com
BUNDLE_ID
com.signapp.sign ← Vérifier cette ligne
PROJECT_ID
sign-app-xxxxx
...
```

■ Vérification critique

Le champ **BUNDLE_ID** dans le fichier doit être **exactement** "com.signapp.sign" — identique au Bundle ID configuré dans Xcode.

Une divergence empêche Firebase de s'initialiser correctement.

3.3 — Ajouter le fichier dans Xcode

Cette étape est cruciale : le fichier doit être ajouté via Xcode (pas juste copié dans le dossier) pour être inclus dans le bundle de l'app lors de la compilation.

1

■ Ouvrir ios/Runner.xcworkspace dans Xcode

Si Xcode n'est pas ouvert : open ios/Runner.xcworkspace

2

■ Localiser le dossier Runner dans l'arbre Xcode

Dans le panneau gauche de Xcode (Navigator), repérer le groupe "Runner".

C'est le dossier bleu principal (pas le projet racine, pas les Pods).

3

■ Ajouter le fichier via clic droit

Clic droit sur le groupe "Runner" → "Add Files to Runner..."

Dans le sélecteur de fichier, naviguer vers votre dossier Téléchargements.

Sélectionner GoogleService-Info.plist.

OPTIONS IMPORTANTES à vérifier :

- Copy items if needed (OBLIGATOIRE)
- Add to targets: Runner (OBLIGATOIRE)

Cliquer "Add".

4

■ Vérifier que le fichier est visible dans Xcode

Dans l'arbre du projet, GoogleService-Info.plist doit apparaître

sous le groupe Runner (avec une icône de fichier plist blanche).

■■ Ne pas utiliser le Finder seul

Copier le fichier dans ios/Runner/ via le Finder sans passer par Xcode ne suffit pas : Xcode ne saura pas l'inclure dans le bundle de compilation. Utilisez TOUJOURS "Add Files to Runner..." depuis Xcode.

3.4 — Vérifier l'intégration Firebase iOS

1

■ Builder le projet en mode Debug

Dans Xcode : Product → Run (■R)

Ou depuis Flutter : `flutter run --debug`

L'app doit démarrer sans crash.

Commande Flutter — Test Debug

```
# Depuis le terminal Mac (projet sign_application) :  
flutter run --debug  
  
# Résultat attendu dans les logs :  
I/flutter (XXXX): Firebase initialisé avec succès  
  
# Ou absence du message d'erreur Firebase  
  
# Si Firebase n'est PAS configuré (mode fallback actif) :  
■■ Firebase non initialisé : [NSBundle mainBundle]...  
# → Le fichier est absent ou mal ajouté dans Xcode
```

2

■ Vérifier dans la Firebase Console

Aller sur console.firebase.google.com → votre projet.

Project Overview → cliquer sur l'app iOS.

Si un device iOS s'est connecté → section "Latest activity" mise à jour.

3

■ Vérifier Crashlytics

Firebase Console → Crashlytics → sélectionner l'app iOS.

Forcer un crash de test (optionnel) :

Test Crashlytics iOS

```
// Dans main.dart (temporaire – retirer après test) :  
FirebaseCrashlytics.instance.crash();  
  
// Résultat attendu : crash visible dans Firebase Console  
// sous Crashlytics → app iOS → dans les 5 minutes.
```

■ Intégration Firebase iOS réussie

Quand tout est correctement configuré :

- L'app démarre sans crash sur iOS
- `firebase_core` est initialisé
- Crashlytics collecte les erreurs en production
- Firebase Analytics enregistre les sessions

04 — Build Release iOS (Archive + Upload)

Une fois BLK-05 et AS-03 résolus, voici la procédure complète pour créer un build release et le soumettre à l'App Store.

1

■ Build depuis Flutter

Depuis le terminal Mac, à la racine du projet :

Terminal Mac — flutter build ipa

```
# Build IPA release :  
flutter build ipa --release \  
--dart-define=API_BASE_URL=https://sign-backend-ha5a.onrender.com/sign  
  
# Si succès :  
Built build/ios/archive/Runner.xcarchive  
Built build/ios/ipa/sign_application.ipa
```

2

■ Ouvrir l'archive dans Xcode Organizer

Depuis Xcode : Window → Organizer

Sélectionner l'archive la plus récente.

Cliquer "Distribute App".

3

■■ Upload vers App Store Connect

Sélectionner "App Store Connect".

Cliquer "Next" → "Upload".

Attendre la fin de l'upload (~2-5 minutes).

4

■ Valider sur App Store Connect

Aller sur <https://appstoreconnect.apple.com>

Sélectionner votre app SIGN.

Dans "TestFlight" ou "iOS App", vérifier que le build est visible.

05 — Checklist finale avant App Store

Configuration Signing (BLK-05)

■	Compte Apple Developer actif (99\$/an)	BLK-05
■	Xcode ouvert sur ios/Runner.xcworkspace	BLK-05
■	Automatically manage signing activé	BLK-05
■	Team sélectionné dans Signing & Capabilities	BLK-05
■	Bundle ID = com.signapp.sign	BLK-05
■	Certificat "Apple Distribution" dans le trousseau	BLK-05

Configuration Firebase iOS (AS-03)

■	App iOS créée dans Firebase Console (com.signapp.sign)	AS-03
■	GoogleService-Info.plist téléchargé	AS-03
■	Fichier ajouté via "Add Files to Runner..." dans Xcode	AS-03
■	"Copy items if needed" coché	AS-03
■	"Add to targets: Runner" coché	AS-03
■	Build Debug sans crash Firebase	AS-03

Déploiement final

■	flutter build ipa --release réussi	BUILD
■	CFBundleName = "SIGN" (corrigé dans l'audit)	AS-04
■	NSPhotoLibraryUsageDescription présent (corrigé)	BLK-03
■	Portrait uniquement configuré (corrigé)	AS-05
■	UIUserInterfaceStyle = Light (corrigé)	AS-06
■	Tests sur device physique iPhone	QA
■	TestFlight distribué à l'équipe	QA
■	Politique de confidentialité hébergée publiquement	APP STORE
■	Screenshots App Store (6.5" + 12.9" iPad)	APP STORE
■	Description app en français (≤4000 car)	APP STORE

Score App Store
après ces 2
corrections

93 /
100

Score Play Store
déjà atteint

88 /
100